

B.Tech. (Computer Science and Engineering)

Semester – VII

Subject: MACHINE LEARNING

Marks: 64

Assignment – 3

✓ SECTION – A [2 × 5 = 10 Marks]

Q1. [CO3] Define gradient descent in the context of machine learning optimization.

Answer:

Gradient Descent is an optimization algorithm used to minimize the cost/loss function in machine learning by iteratively updating model parameters in the direction of the **negative gradient**.

$$\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Where α = learning rate.

Q2. [CO3] Mention two key differences between supervised and unsupervised learning.

Answer:

Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data
Output is known (classification, regression)	Output is unknown (clustering, PCA)

Q3. [CO3] What is the role of learning rate in training machine learning models?

Answer:

The learning rate controls **how big the step size** is while updating weights during training.

- Too high → model overshoots minimum and diverges
 - Too low → model trains very slowly and may get stuck
-

Q4. [CO3] Give two reasons why feature normalization is important before model training.

Answer:

1. Ensures all features are on a similar scale, improving convergence speed.
2. Prevents large-valued features from dominating distance-based algorithms like KNN, SVM.

Q5. [CO3] What is the purpose of bias in a neuron of a neural network?

Answer:

The bias allows the activation function to shift left or right, enabling the model to fit data **even when input = 0**. It increases model flexibility.

✓ **SECTION – B [3 × 7 = 21 Marks]**

Q1. [CO3] Explain the concept of loss functions in machine learning. Discuss commonly used loss functions for regression and classification tasks.

A **loss function** measures how well a model is performing by calculating the difference between predicted and actual values. The goal is to minimize this loss.

✓ **Common Loss Functions:**

Task	Loss Function	Formula
Regression	Mean Squared Error (MSE)	$\frac{1}{m} \sum (y - \hat{y})^2$
Regression	Mean Absolute Error (MAE)	$(\frac{1}{m} \sum$
Classification	Binary Cross Entropy	$-[y \log (\hat{y}) + (1 - y) \log (1 - \hat{y})]$
Classification	Categorical Cross Entropy	$-\sum y_i \log (\hat{y}_i)$

Loss is minimized using optimization methods like gradient descent.

Q2. [CO4] Describe the architecture and working of a simple feedforward neural network. How are weights and biases updated during learning?

Answer:

✓ **Architecture**

Input Layer → Hidden Layer(s) → Output Layer

Each neuron performs weighted sum + activation:

$$z = W \cdot X + b, a = f(z)$$

✓ Working

1. Forward propagation → computes predicted output
2. Loss is calculated
3. Backpropagation computes gradients
4. Weights updated using gradient descent:

$$W := W - \alpha \frac{\partial J}{\partial W}$$

Q3. [CO3] What is cross-validation? Discuss its types and importance in improving model reliability.

Answer:

Cross-validation is a technique used to evaluate a model by training and testing it on different subsets of data to avoid overfitting.

✓ Types of Cross-Validation

Type	Description
k-Fold CV	Dataset split into k equal parts, repeated k times
Stratified k-Fold	Preserves class distribution (used in classification)
Leave-One-Out (LOOCV)	k = n (each sample used once as test data)
Hold-out Method	Simple train-test split

✓ Importance:

- ✓ Gives better performance estimation
 - ✓ Reduces bias from single train-test split
 - ✓ Helps in hyperparameter tuning
-

✓ SECTION – C [3 × 11 = 33 Marks]

Q1. [CO4] Explain in detail how the backpropagation algorithm operates in multilayer neural networks. Include the mathematical derivation and explain the role of partial derivatives.

Answer:

Backpropagation is the learning algorithm used in multilayer neural networks to update weights based on the error calculated at the output. It uses the **chain rule of calculus** to compute gradients layer-by-layer.

✔ Steps in Backpropagation

Phase	Description
1. Forward Propagation	Compute predicted output using weights and activation functions
2. Loss Calculation	Compute error using loss function (e.g., MSE, cross entropy)
3. Backward Propagation	Compute gradient of loss w.r.t weights using chain rule
4. Weight Update	Apply gradient descent to update parameters

✔ Mathematical Derivation

Let:

- Activation of neuron = $a = f(z)$
- Weighted sum = $z = W \cdot X + b$
- Cost function = $J(\theta)$

Gradient descent rule:

$$W := W - \alpha \frac{\partial J}{\partial W}$$

✔ Error Term (Delta Rule)

For output layer:

$$\delta^{(L)} = (a^{(L)} - y) \cdot f'(z^{(L)})$$

For hidden layers:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \cdot f'(z^{(l)})$$

✔ Weight Update Formula

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

$$W^{(l)} := W^{(l)} - \alpha \frac{\partial J}{\partial W^{(l)}}$$

✓ Role of Partial Derivatives

They tell us **how much the loss changes** with respect to each weight.
If derivative is **high** → **weight changes more**, if **low** → **small update**.

✓ Advantages

- ✓ Works with multilayer networks
- ✓ Efficient (uses matrix calculus)
- ✓ Basis of deep learning (CNN, RNN, etc.)

✓ Limitations

- ✗ Slow when data is large
 - ✗ Can get stuck in local minima
 - ✗ Suffers from vanishing gradient problem
-

Q2. [CO4] Discuss the concept of Regularization in depth. Compare L1 and L2 regularization and explain how each helps to reduce overfitting.

Answer:

Regularization is a technique used to **reduce overfitting** by adding a **penalty term** to the loss function that discourages large weight values.

✓ Why Regularization?

- Overfitting occurs when model learns noise & fits training data too well
 - Regularization **simplifies the model** by shrinking weights
 - Helps improve **generalization** on unseen data
-

✓ Types of Regularization

1 L2 Regularization (Ridge)

Adds squared weights to cost function:

$$J(\theta)_{L2} = J(\theta) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- ✓ Shrinks weights smoothly
 - ✓ Does **not** make weights zero
 - ✓ Good when all features are useful
-

2 L1 Regularization (Lasso)

Adds absolute value of weights:

$$J(\theta)_{L1} = J(\theta) + \frac{\lambda}{m} \sum_{j=1}^n |\theta_j|$$

- ✓ Can make some weights **exactly zero**
 - ✓ Performs **feature selection**
 - ✓ Useful when dataset has many irrelevant inputs
-

✓ Comparison Table

Feature	L1 (Lasso)	L2 (Ridge)
Penalty	(	W
Effect	Sparse weights (some zero)	All weights shrink
Feature selection	Yes	No
Best for	High-dimensional data	Multicollinearity
Optimization	Non-smooth	Smooth, differentiable

✓ Effect of λ (Regularization Strength)

λ value	Effect
$\lambda = 0$	No regularization → risk of overfitting
λ small	Slight shrinkage
λ large	Strong shrinkage → underfitting possible

Q3. [CO4] Define Principal Component Analysis (PCA) and describe step-by-step how PCA reduces dimensionality. Illustrate with an example.

Answer:

PCA is a **dimensionality reduction technique** that converts high-dimensional data into fewer new variables called **principal components**, while preserving maximum variance.

✅ Steps of PCA

1. **Standardize the data** (mean = 0, variance = 1)
2. **Compute covariance matrix**

$$Cov(X) = \frac{1}{m} X^T X$$

3. **Find eigenvalues and eigenvectors**
4. **Sort eigenvectors in descending order of eigenvalues**
5. **Select top k eigenvectors → principal components**
6. **Transform data into new reduced dimensions**

$$Z = X \cdot W_k$$

✅ Example

Dataset (2D):

(2,3), (4,5), (3,6), (5,8)

After PCA → 1D projection retaining 95% variance

Data reduces from **2 features** → **1 feature**, but most information remains.

✅ Advantages

- ✓ Removes redundant and correlated features
- ✓ Speeds up ML algorithms
- ✓ Visualization of high-dimensional data

✅ Limitations

- ✗ Principal components are not interpretable
- ✗ Data must be linear
- ✗ Loses some information